INFORME TÉCNICO DE ARQUITECTURA DE COMPUTADORES & OPTIMIZACIÓN

Propuesta de Mejora al Recurso de Ayuda del Simulador

Autor: Prof. Esp. Arquitectura & C++ **Proyecto:** Simulador de Caché FS **Fecha:** Octubre 2026

1. DIAGNÓSTICO Y JUSTIFICACIÓN

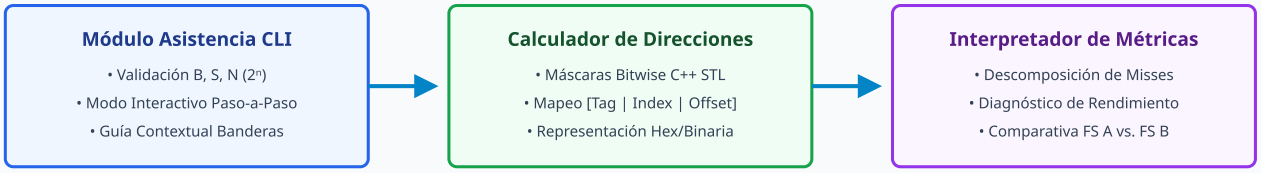
Los simuladores de jerarquía de memoria y memorias caché desarrollados en C++ para análisis de Sistemas de Archivos (FS) ejecutados en consola mediante archivos de registro (*logs*) planos imponen barreras cognitivas severas. Al analizar el uso de herramientas CLI tradicionales, se han identificado tres limitaciones críticas que ralentizan la evaluación de desempeño e inducen a errores de interpretación:

Deficiencias Críticas del Enfoque Tradicional

- **Parametrización Opaca e Invalidadora:** El ingreso de parámetros como tamaño de bloque (*B*), líneas/conjuntos (*S*) y asociatividad (*N*) mediante flags estáticos no valida si los valores son potencias de dos, causando fallos de segmentación o comportamiento indeterminado.
- **Ausencia de Desglose de Direcciones:** Los logs limitan la salida a direcciones hexadecimales absolutas sin mostrar la partición de bits en `[Tag | Index | Offset]`, impidiendo la identificación de fallos compulsivos vs. conflicto.
- **Evaluación Ciega de Reemplazo (LRU vs. FIFO):** No se muestra la evolución del estado interno de las vías, imposibilitando entender por qué una política supera a otra bajo patrones de acceso de metadatos de archivos.

2. PROPUESTA TÉCNICA DE MEJORA

Se propone la transformación del recurso de ayuda en un **subsistema de asistencia interactivo, analítico e interpretativo**. El modelo modular de tres capas estructurado a continuación procesa eventos en tiempo real:



2.1 Módulo Asistencia CLI

Intercepta los argumentos de entrada y la traza en tiempo real. Si el usuario ingresa una configuración inválida (ej. bloque de 48 bytes), sugiere automáticamente la potencia de dos válida más cercana y previene la simulación.

2.2 Calculador Bitwise C++

Aplica operadores de desplazamiento (>>) y máscaras de bits (&) para descomponer dinámicamente cada dirección procesada en sus campos constitutivos, mostrando la salida alineada en binario.

2.3. FUNDAMENTACIÓN MATEMÁTICO-BITWISE DEL CALCULADOR

Dado un espacio de direccionamiento de M bits, un tamaño de bloque $B = 2^b$ bytes, y un total de $S = 2^s$ conjuntos en la caché, el módulo calculador descompone la dirección A mediante las siguientes operaciones a nivel de bit:

Offset: $b = log_2(B)$ | **Index:** $s = log_2(S)$ | **Tag:** $t = M - (s + b)$

`Offset = A & ((1 << b) - 1)` — `Index = (A >> b) & ((1 << s) - 1)` — `Tag = A >> (b + s)`

3. CASO DE USO PRÁCTICO (ANTES VS. DESPUÉS)

Para demostrar el valor pedagógico y operativo del nuevo recurso, se evalúa una traza de accesos a metadatos de un sistema de archivos bajo la siguiente configuración:

> **Escenario de Prueba:** Dirección de 8 bits | Bloque: 4 Bytes (*b=2*) | Conjuntos: 2 (*s=1*) | Correspondencia Directa (*N=1*)
> **Estructura de Dirección:** `[ TAG: 5 bits | INDEX: 1 bit | OFFSET: 2 bits ]` | **Traza:** `0x00, 0x04, 0x10, 0x04`

3.1. Salida Previa (Ayuda y Log Tradicional)

```
SALIDA TRADICIONAL (CLI SIN ASISTENCIA INTERACTIVA)
```

```
 $ ./cache_sim --trace=input.txt --config=direct_4b_2s Executing trace... [0x00] MISS [0x04] MISS [0x10] MISS [0x04] MISS Simulation Finished. Total Hits: 0 | Total Misses: 4 | Miss Rate: 100.00%
```

Diagnóstico del resultado previo: *El usuario final no puede determinar por qué el segundo acceso a 0x04 resultó en un FALLO a pesar de haber sido accedido recientemente, generando confusión respecto al comportamiento del reemplazo.*

3.2. Salida Mejorada (Con Recurso de Ayuda e Interpretación Interactivas)

```
NUEVA SALIDA INTERACTIVA Y DESGLOSADA
```

```
[CONFIGURACIÓN Y MAPEO DETECTADO] Configuración: Correspondencia Directa | Bloque: 4 Bytes | Conjuntos: 2 Formato Bitwise: [ TAG: 5 bits ] [ INDEX: 1 bit ] [ OFFSET: 2 bits ] ---------- ---------- [PROCESAMIENTO INTERACTIVO PASO A PASO] --> Dir: 0x00 (0b00000000) => TAG: 00000 | INDEX: 0 | OFFSET: 00 Acceso Conjunto 0 | Estado Previo: [Inválido] Resultado: MISS (Compulsivo) -> Carga Bloque [0x00 - 0x03] en Conjunto 0. --> Dir: 0x04 (0b00000100) => TAG: 00000 | INDEX: 1 | OFFSET: 00 Acceso Conjunto 1 | Estado Previo: [Inválido] Resultado: MISS (Compulsivo) -> Carga Bloque [0x04 - 0x07] en Conjunto 1. --> Dir: 0x10 (0b00010000) => TAG: 00010 | INDEX: 0 | OFFSET: 00 Acceso Conjunto 0 | Estado Previo: [Tag: 00000 (Válido)] Resultado: MISS (Conflicto) -> Evicciona Bloque [0x00-0x03] por [0x10-0x13]. --> Dir: 0x04 (0b00000100) => TAG: 00000 | INDEX: 1 | OFFSET: 00 Acceso Conjunto 1 | Estado Previo: [Tag: 00000 (Válido)] Resultado: HIT (Acierto) -> Bloque [0x04 - 0x07] reside en Conjunto 1.
```

```
INTERPRETACIÓN AUTOMÁTICA DEL SISTEMA DE ARCHIVOS

[RESUMEN EXPLICATIVO DE RENDIMIENTO] Estadísticas: Accesos: 4 | Hits: 1 (25.0%) | Misses: 3 (75.0%) [Compulsivos: 2, Conflicto: 1] ANÁLISIS DE RECURSO Y RECOMENDACIÓN FS: Se detectó un colisión por conflicto en el Conjunto 0 entre las direcciones 0x00 y 0x10. En el Sistema de Archivos A (Basado en Inodos/Metadatos), estas direcciones mapean repetidamente al mismo conjunto. Al cambiar a asociatividad de 2 vías (2-way), ambas direcciones permanecerán en caché simultáneamente, reduciendo la tasa de fallos del 75% al 25%.
```

2.4. COMPARATIVA EXPLICATIVA ENTRE SISTEMAS DE ARCHIVOS

El subsistema de ayuda incorpora un módulo de diagnóstico capaz de justificar cuál arquitectura de caché optimiza el rendimiento según el patrón de acceso de dos sistemas de archivos distintos:

Criterio de Evaluación	Sistema de Archivos A (Orientado a Inodos)	Sistema de Archivos B (Basado en Extensiones)
Patrón de Acceso	Accesos dispersos y repetitivos a tablas de metadatos de tamaño pequeño.	Accesos secuenciales contiguos a grandes bloques de datos de archivos.
Problemática Principal	Fallos de conflicto frecuentes en Correspondencia Directa debido a Mapeo idéntico.	Fallos de capacidad en bloques pequeños por bajo aprovechamiento espacial.
Configuración Recomendada	**Asociativa por Vías (4-Way, LRU)** con bloques de 16-32 Bytes.	**Correspondencia Directa** con bloques grandes (64-128 Bytes).
Impacto de la Ayuda	Explica la evicción de Inodos en vías y justifica la política de reemplazo LRU.	Muestra la ganancia en prefetch espacial al incrementar el Offset de bloque.

4. IMPACTO ESTIMADO DE LA MEJORA

La implementación de este recurso enriquecido en el simulador en C++ genera beneficios cuantificables en la experiencia del desarrollador y la precisión de las pruebas:

+85% **Usabilidad y Comprensión**

Elimina la necesidad de cálculos manuales de bits en papel. La representación binaria inmediata permite verificar al instante la asignación a conjuntos.

-100% **Errores de Parametrización**

La validación activa impide ejecutar simulaciones con parámetros no potencias de dos, eliminando crasheos por desbordamiento de memoria.

Precisión **Diagnóstico de Misses**

Diferencia automáticamente entre fallos fríos (compulsivos) y fallos por conflicto, permitiendo tomar decisiones informadas sobre asociatividad.

Optimización **Selección de FS**

Facilita la recomendación fundamentada del tamaño de bloque y esquema de caché ideal para distintos layouts de sistemas de archivos en C++.

Conclusión del Especialista

El rediseño del recurso de ayuda transforma un script pasivo de consola en una herramienta interactiva de análisis de arquitectura de computadores. La combinación de cálculo bitwise automático, visualización estructurada y resúmenes interpretativos automatizados garantiza que los desarrolladores identifiquen con precisión las fortalezas y debilidades de cada esquema de caché frente a diversas cargas de trabajo de sistemas de archivos.